
Lab Assignment # 17

Name of Student : Chathraboina Sagar
Enrollment No. : 2303A51522
Batch No. : 22

Task 1 – Password Security Enhancement

Task: Analyze an AI-generated Python program that stores passwords in plain text and improve its security.

Instructions:

- Prompt AI to generate a simple user registration script that stores usernames and passwords.
- Check whether passwords are stored as plain text.
- Identify the security risks associated with plain-text password storage.
- Refactor the program to use secure password hashing (e.g., hashlib or bcrypt).
- Test the updated script to ensure proper authentication functionality.

Expected Output:

A secure Python script that stores hashed passwords instead of plain text and successfully validates user login credentials.

Prompt:-

Generate a Python program for user registration and login that stores usernames and passwords. First, create a basic version that stores passwords in plain text. Then, modify the program to securely store passwords using hashing techniques like hashlib or bcrypt. Ensure the program supports user authentication by verifying hashed passwords during login.

Code:-

```
# Task-01
# Generate a python code that cleans and prepare a students academic dataset
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Load dataset
data = pd.read_csv(r"C:\AIAC LAB\students_cleaned.csv")
print("Original data shape:", data.shape)
print(data.head())

# Handle missing values
data['marks'].fillna(data['marks'].mean(), inplace=True)
data['attendance'].fillna(data['attendance'].mean(), inplace=True)
data['department'].fillna(data['department'].mode()[0], inplace=True)

# Remove duplicates
data.drop_duplicates(inplace=True)

# Standardize text fields
data['department'] = data['department'].str.strip().str.title()
data['gender'] = data['gender'].str.strip().str.lower()

# Encode categorical variables
le_gender = LabelEncoder()
le_department = LabelEncoder()

data['gender_encoded'] = le_gender.fit_transform(data['gender'])
data['department_encoded'] = le_department.fit_transform(data['department'])

# Display cleaned data
print("\nCleaned data shape:", data.shape)
print(data.head())
print("\nData types:\n", data.dtypes)
print("\nMissing values:\n", data.isnull().sum())
```

Output:-

```
Cleaned data shape: (4, 8)
  student_id  name gender department marks attendance gender_encoded department_encoded
0          1  Alice    f  Computer Science  85.0      95.0           0              0
1          2   Bob    m   Electronics  88.5      85.0           1              1
2          3 Charlie  NaN  Computer Science  92.0      90.0           2              0
3          4  David    m  Mechanical Engineering  88.5      90.0           1              2

Data types:
  student_id      int64
  name            str
  gender          str
  department      str
  marks           float64
  attendance      float64
  gender_encoded  int64
  department_encoded  int64
  dtype: object

Missing values:
  student_id      0
  name            0
  gender          1
  department      0
  marks           0
  attendance      0
  gender_encoded  0
  department_encoded  0
  dtype: int64
```

Justification:

The updated prompt focuses on identifying and fixing a critical security flaw—plain-text password storage. By introducing hashing techniques, the program ensures that sensitive user credentials are protected even if the database is compromised. This enhances data security and follows best practices in authentication systems.

Task 2 – Secure API Key Management

Task: Evaluate an AI-generated script that uses hardcoded API keys and improve its security.

Instructions:

- Ask AI to generate a Python script that connects to an external API using a hardcoded API key.
- Identify the risks of embedding sensitive credentials directly in the source code.
- Refactor the program to use environment variables for storing API keys securely.
- Demonstrate that the modified script successfully retrieves data without exposing credentials.

Expected Output:

A revised script that securely accesses API keys through environment variables instead of hard coding them.

Prompt:-

Generate a Python script that connects to an external API using an API key. First, demonstrate a version where the API key is hardcoded in the script. Then, refactor the code to securely store and access the API key using environment variables. Ensure the script successfully retrieves data without exposing sensitive credentials.

Code:-

```
# Task-02
# Generate a python code that performs preprocess financial transaction data. convert tran
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Load dataset
transactions = pd.read_csv(r"C:\AIAC LAB\transactions_raw.csv")
print("Original data shape:", transactions.shape)
print(transactions.head())

# Convert to datetime format
transactions['transaction_date'] = pd.to_datetime(transactions['transaction_date'])

# Derive new features
transactions['transaction_month'] = transactions['transaction_date'].dt.month
transactions['transaction_day_of_week'] = transactions['transaction_date'].dt.day_name()
transactions['transaction_year'] = transactions['transaction_date'].dt.year

# Handle extreme values by capping at 95th percentile
percentile_95 = transactions['amount'].quantile(0.95)
transactions['amount'] = transactions['amount'].clip(upper=percentile_95)

# Normalize transaction amounts
scaler = MinMaxScaler()
transactions['amount_normalized'] = scaler.fit_transform(transactions[['amount']])

# Display preprocessed data
print("\nPreprocessed data shape:", transactions.shape)
print(transactions.head())
print("\nData types:\n", transactions.dtypes)
print("\nMissing values:\n", transactions.isnull().sum())
```

Output:-

```
Original data shape: (6, 4)
  transaction_id customer_id  amount transaction_date
0          T001         C001   100.5      2025-01-15
1          T002         C002   250.0      2025-02-20
2          T003         C001    50.0      2025-01-10
3          T004         C003  10000.0      2025-03-05
4          T005         C002   175.5      2025-02-25

Preprocessed data shape: (6, 8)
  transaction_id customer_id  amount transaction_date  transaction_month  transaction_day_of_week  transaction_year  amount_normalized
0          T001         C001   100.5      2025-01-15                1             Wednesday            2025             0.006722
1          T002         C002   250.0      2025-02-20                2              Thursday            2025             0.026622
2          T003         C001    50.0      2025-01-10                1              Friday            2025             0.000000
3          T004         C003  7562.5      2025-03-05                3             Wednesday            2025             1.000000
4          T005         C002   175.5      2025-02-25                2              Tuesday            2025             0.016705
```

```
Data types:
transaction_id      str
customer_id         str
amount              float64
transaction_date     datetime64[us]
transaction_month    int32
transaction_day_of_week str
transaction_year     int32
amount_normalized    float64
dtype: object

Missing values:
transaction_id      0
customer_id         0
amount              0
transaction_date     0
transaction_month    0
transaction_day_of_week 0
transaction_year     0
amount_normalized    0
dtype: int64
```

Justification:

Embedding API keys directly in the source code is highly insecure because the code may be shared or uploaded to public repositories, making the credentials visible to others. This can lead to unauthorized use of services, financial loss, and data security issues. Using environment variables allows sensitive information to be stored outside the code, reducing the risk of exposure. It also improves flexibility by enabling different

Task 3 – Weak Encryption Detection

Task:Examine an AI-generated program that uses weak encryption techniques and strengthen it.

Instructions:

- Prompt AI to generate a simple encryption-decryption program using a basic or outdated method.
- Identify weaknesses in the encryption approach (e.g., reversible encoding or outdated algorithms).
- Refractor the program using stronger encryption techniques available in Python libraries.
- Validate that encrypted data cannot be easily reversed without the correct key.

Expected Output:

An improved encryption program implementing stronger and more secure cryptographic practices.

Prompt:-

Generate a Python program that performs encryption and decryption using a simple method such as base64 or Caesar cipher. Then refactor the program using a stronger encryption technique like the cryptography library (Fernet). Ensure that encrypted data cannot be easily reversed without the correct key.

Code:-

```
# Task-03
# Generate a python code that clean and preprocess environmental sensor data. handle
import pandas as pd
import numpy as np

# Load dataset
data = pd.read_csv(r"C:\AIAC LAB\sensor_data_raw_detailed.csv")
print("Original data shape:", data.shape)
print(data.head())

# Handle missing values using interpolation
data['temperature'] = data['temperature'].interpolate(method='linear')
data['humidity'] = data['humidity'].interpolate(method='linear')
data['air_quality_index'] = data['air_quality_index'].interpolate(method='linear')

# Remove duplicates
data.drop_duplicates(inplace=True)

# Standardize text fields
data['sensor_location'] = data['sensor_location'].str.strip().str.title()

# Convert timestamp to datetime
data['timestamp'] = pd.to_datetime(data['timestamp'])

# Derive day/night feature based on hour
data['hour'] = data['timestamp'].dt.hour
data['day_night'] = data['hour'].apply(lambda x: 'Night' if 6 <= x < 18 else 'Day')

# Display cleaned data
print("\nCleaned data shape:", data.shape)
print(data.head())
print("\nData types:\n", data.dtypes)
print("\nMissing values:\n", data.isnull().sum())
```

Output:-

```
traffic = pd.read_csv(r"C:\AIAC LAB\traffic_data_raw.csv")
Original data shape: (208, 6)
   timestamp  sensor_id  sensor_location  temperature  humidity  air_quality_index
0  2025-01-01 00:00:00      S3          NaN      24.627189    81.271645      37.489499
1  2025-01-01 00:30:00      S4    Residential      16.256397         NaN      59.092204
2  2025-01-01 01:00:00      S1          NaN      17.266938    57.209033      42.947044
3  2025-01-01 01:30:00      S3          NaN      23.979983    54.817752      81.036983
4  2025-01-01 02:00:00      S3  Industrial Area         NaN    53.765060      42.451622

Cleaned data shape: (205, 8)
   timestamp  sensor_id  sensor_location  temperature  humidity  air_quality_index  hour  day_night
0  2025-01-01 00:00:00      S3          NaN      24.627189    81.271645      37.489499      0      Day
1  2025-01-01 00:30:00      S4    Residential      16.256397    69.240339      59.092204      0      Day
2  2025-01-01 01:00:00      S1          NaN      17.266938    57.209033      42.947044      1      Day
3  2025-01-01 01:30:00      S3          NaN      23.979983    54.817752      81.036983      1      Day
4  2025-01-01 02:00:00      S3  Industrial Area      21.615641    53.765060      42.451622      2      Day

Data types:
timestamp          datetime64[us]
sensor_id          str
sensor_location    str
temperature        float64
humidity           float64
air_quality_index  float64
hour               int32
day_night          str
dtype: object

Missing values:
timestamp          0
sensor_id          0
sensor_location    31
temperature        0
humidity           0
2  2025-01-01 01:00:00      S1          NaN      17.266938    57.209033      42.947044      1      Day
3  2025-01-01 01:30:00      S3          NaN      23.979983    54.817752      81.036983      1      Day
4  2025-01-01 02:00:00      S3  Industrial Area      21.615641    53.765060      42.451622      2      Day
```

```
Data types:
timestamp          datetime64[us]
sensor_id           str
sensor_location     str
temperature         float64
humidity            float64
air_quality_index   float64
hour                int32
day_night           str
dtype: object

Missing values:
timestamp          0
sensor_id          0
sensor_location    31
temperature        0
humidity           0
Data types:
timestamp          datetime64[us]
sensor_id           str
sensor_location     str
temperature         float64
humidity            float64
air quality index   float64
```

```
Missing values:
timestamp          0
sensor_id          0
sensor_location    31
temperature        0
humidity           0
timestamp          0
sensor_id          0
sensor_location    31
temperature        0
humidity           0
temperature        0
humidity           0
humidity           0
air_quality_index  0
hour               0
hour               0
day_night          0
dtype: int64
```

Justification:

Simple encryption techniques like base64 encoding or Caesar cipher are not secure because they can be easily decoded using basic methods. These techniques do not provide real protection for sensitive data. By replacing them with strong encryption methods provided by libraries such as `cryptography` (e.g., Fernet), the data becomes securely protected using advanced algorithms and secret keys. Strong encryption ensures that only authorized users with the correct key can access the data. It also protects against common security threats and ensures confidentiality and integrity. This upgrade is essential for building secure applications that handle sensitive information. .

Task 4 – Real-Time Application: Security Review of AI-Generated Web Service

Scenario:

Students select an AI-generated web service snippet (e.g., REST API, file upload handler, or authentication module).

Instructions:

- Perform a structured security review of the generated code.
- Identify at least three potential vulnerabilities.
- Prompt AI to recommend secure coding practices and mitigation strategies.
- Implement the suggested improvements and test the secured version.
- Document the differences between the insecure and secured implementations.

Expected Output:

A security-reviewed and improved version of the selected web service code, with clearly documented vulnerabilities and applied fixes.

Prompt:-

Generate a Python-based web service such as a REST API or file upload handler. Identify security vulnerabilities like improper input validation, lack of authentication, or insecure data handling. Then improve the code by implementing secure coding practices and compare both versions. .

Code:-

```
# Task-04
# Generate a python code a smart city system that collects traffic data from various sensors
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# Load dataset
traffic = pd.read_csv(r"C:\AIAC LAB\traffic_data_raw.csv")
print("Original data shape:", traffic.shape)
print(traffic.head())
print("\nOriginal data summary:\n", traffic.describe())

# Standardize text fields
traffic['road_name'] = traffic['road_name'].str.strip().str.title()
traffic['location'] = traffic['location'].str.strip().str.title()

# Fill missing traffic density values using mean imputation
traffic['traffic_density'].fillna(traffic['traffic_density'].mean(), inplace=True)

# Remove duplicates
traffic.drop_duplicates(inplace=True)

# Normalize speed and vehicle count
scaler = MinMaxScaler()
traffic['speed_normalized'] = scaler.fit_transform(traffic[['speed']])
traffic['vehicle_count_normalized'] = scaler.fit_transform(traffic[['vehicle_count']])

# Display preprocessed data
print("\nCleaned data shape:", traffic.shape)
print(traffic.head())
print("\nPreprocessed data summary:\n", traffic.describe())
print("\nData types:\n", traffic.dtypes)
print("\nMissing values:\n", traffic.isnull().sum())

# Quality comparison summary
print("\n--- Data Quality Summary ---")
print(f"Rows removed (duplicates): {traffic.shape[0]}")
print("Preprocessing completed successfully!")
```

Output:-

```
posts = pd.read_csv('C:\Users\B\Desktop\social_media_posts.csv')
Original data shape: (8, 6)
   timestamp      road_name  location_name  speed  vehicle_count  traffic_density
0  2025-01-01 08:00        NH 44      City Center    65.5           120           High
1  2025-01-01 08:15         nh 44     city center    45.2           180          Very High
2  2025-01-01 08:30  State Highway 1  Suburban Area    72.1            85            Low
3  2025-01-01 08:45         nh44  Industrial Zone     NaN            95          Medium
4  2025-01-01 09:00         NH 44  University Road    58.3           150           High

Original data summary:
      speed  vehicle_count
count  7.000000      8.000000
mean   57.828571     135.000000
std    12.827277      45.747756
min    35.800000      85.000000
25%    51.750000     106.250000
50%    62.400000     120.000000
75%    65.500000     157.500000
max    72.100000     220.000000
```

Justification:

Web services are frequently targeted by attackers due to their exposure to external users. Common vulnerabilities include lack of input validation, weak authentication mechanisms, and improper handling of sensitive data. These issues can lead to attacks such as SQL injection, data breaches, and unauthorized access. By performing a structured security review, these vulnerabilities can be identified and fixed. Implementing secure coding practices such as validating inputs, enforcing authentication, and handling errors properly strengthens the application. This not only improves system security but also ensures reliability and user trust in the application.

Task 5 – Exception Handling Improvement

Task:

Analyze an AI-generated Python program that does not handle runtime errors and improve its reliability.

Instructions:

- Prompt AI to generate a simple program that takes user input and performs arithmetic operations.
- Identify possible runtime errors (e.g., invalid input, division by zero).
- Modify the program to include proper try-except blocks.
- Provide meaningful error messages for invalid inputs.
- Test the improved version with valid and invalid test cases.

Expected Output:

A revised Python program with proper exception handling that prevents crashes and displays user-friendly error messages for invalid inputs.

Prompt:-

Generate a Python program that takes user input and performs arithmetic operations such as addition, subtraction, multiplication, and division. Then enhance the program by adding proper exception handling using try-except blocks to handle errors like invalid input and division by zero. Display meaningful error messages to the user.

Code:-

```
# Task-05
# Generate a python code to clean and preprocess Social media text data for
import pandas as pd
import re

# Load dataset
posts = pd.read_csv(r"C:\AIAC LAB\social_media_posts.csv")
print("Original data shape:", posts.shape)
print(posts.head())

# Remove duplicates and null entries
posts.drop_duplicates(inplace=True)
posts = posts.dropna(subset=['text'])

# Define stop words manually (common English stop words)
stop_words = {
    'a', 'an', 'and', 'are', 'as', 'at', 'be', 'by', 'for', 'from',
    'has', 'he', 'in', 'is', 'it', 'its', 'of', 'on', 'or', 'that',
    'the', 'to', 'was', 'will', 'with', 'i', 'me', 'my', 'you', 'your'
}

# Function to clean text
def clean_text(text):
    # Remove URLs
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)
    # Remove special characters and extra spaces
    text = re.sub(r'^a-zA-Z\s]', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    # Convert to lowercase
    text = text.lower()
    # Tokenize and remove stop words
    tokens = text.split()
    tokens = [word for word in tokens if word not in stop_words]
    return ' '.join(tokens)

# Apply cleaning
posts['text_cleaned'] = posts['text'].apply(clean_text)

# Display cleaned data
print("\nCleaned data shape:", posts.shape)
print(posts.head())
print("\nData types:\n", posts.dtypes)
print("\nMissing values:\n", posts.isnull().sum())
```

Output:-

```
Original data shape: (10, 2)
  post_id  text
0      1  Love this new update! #GreatApp https://exampl...
1      2    This app is terrible 🤬 Never working properly!
2      3  Love this new update! #GreatApp https://exampl...
3      4                                     NaN
0      1  Love this new update! #GreatApp https://exampl...
1      2    This app is terrible 🤬 Never working properly!
2      3  Love this new update! #GreatApp https://exampl...
3      4                                     NaN
1      2    This app is terrible 🤬 Never working properly!
2      3  Love this new update! #GreatApp https://exampl...
3      4                                     NaN
4      5  Check out my new profile pic! Looking good 🍷

Cleaned data shape: (9, 3)
  post_id  text  text_cleaned
4      5  Check out my new profile pic! Looking good 🍷
```

```
Cleaned data shape: (9, 3)
```

	post_id	text	text_cleaned
0	1	Love this new update! #GreatApp https://exampl...	love this new update greatapp
1	2	This app is terrible 🤬 Never working properly!	this app terrible never working properly
2	3	Love this new update! #GreatApp https://exampl...	love this new update greatapp
0	1	Love this new update! #GreatApp https://exampl...	love this new update greatapp
1	2	This app is terrible 🤬 Never working properly!	this app terrible never working properly
2	3	Love this new update! #GreatApp https://exampl...	love this new update greatapp
4	5	Check out my new profile pic! Looking good 🔥	check out new profile pic looking good
5	6	Why is everyone hating on this? It's fine...	why everyone hating this fine

```
Data types:
  post_id      int64
  text         str
  text_cleaned str
dtype: object
```

```
Missing values:
  post_id      0
  text         0
  text_cleaned 0
dtype: int64
```

Justification:

Programs without proper error handling are prone to unexpected crashes when invalid inputs or runtime errors occur. This can lead to poor user experience and unreliable software behavior. By implementing exception handling using try-except blocks, the program can detect and manage errors effectively without terminating abruptly. It allows the program to provide clear and meaningful feedback to users, helping them correct their mistakes. Additionally, it improves code reliability, maintainability, and debugging. Exception handling is an essential feature in robust software development, ensuring smooth execution even in unexpected situations.